1. Executive Summary

The GTM Lead Intelligence Agent finds enterprise buying opportunities in public news and press releases and turns them into scored leads for Trianz sales teams. A user sets the regions, industries, and Concierto offerings they care about. The agent builds search queries from buying signals, pulls articles from three sources, uses an AI model to decide whether each article points to a real buyer, scores the opportunity, and saves the qualified leads. Leads appear in the portal, and a digest email summarizes new leads after each run.

The system is a single Next.js application. Discovery, scheduling, AI analysis, and the API all run in one process, backed by PostgreSQL. It runs on Azure App Service with Azure OpenAI for analysis and Azure Communication Services for email.

2. Overview and Problem

The problem

Sales teams find new opportunities by reading news by hand. Most of the useful signal, such as a data center exit, a FinOps program, or a new AI initiative, appears in press releases and trade coverage days before it reaches account teams. Manual monitoring is slow, uneven across regions, and easy to miss.

What the agent does

- Converts each user's preferences and the active buying signals for their offerings into news search queries.
- Collects articles, removes duplicates, and analyzes each one with an AI model.
- Keeps only articles that describe an enterprise buyer, a matching offering, and an allowed region.
- Scores each lead as Very Hot, Hot, or Warm, from 0 to 100.
- Shows the leads in the portal and sends a digest email.

Who uses it

User	How they use it
Sales reps and account teams	Review leads for their regions and industries, open source articles, and follow up.
Sales leadership	Track the Very Hot and Hot pipeline and read the daily or weekly digest.
Offering owners	Maintain the buying signals that drive searches for their product.

What it returns

Each lead includes the company, industry, country, the buying signal detected, the Concierto offering it maps to, a score, a one-line summary, the reason it qualified, project stage, business impact, procurement activity, and a link to the source article.

3. Technology Stack

Layer	Technology	Purpose

Web framework	Next.js 16.3 with the App Router, Turbopack in development	Pages, API route handlers, and the server runtime
Language	TypeScript	All application code
UI	React 19, Tailwind CSS 4, shadcn-style components	Portal screens
Authentication	NextAuth v4 with the Azure AD provider	Microsoft sign-in and sessions
Database	PostgreSQL 16 (Azure Database for PostgreSQL in production)	Users, preferences, signals, runs, articles, leads
Database client	pg connection pool	Queries from repositories
AI analysis	Azure OpenAI, gpt-4o deployment	Extracting and scoring leads
Fallback analysis	Local keyword analyzer	Used when Azure OpenAI is not configured
News sources	Google News, PR Newswire, Business Wire RSS	Article collection
Email	Azure Communication Services Email SDK	Digest emails
Digest feed	GET `/api/digest` HTML	Read by Power Automate for forwarding
Hosting	Azure App Service (Linux, Node runtime)	Production hosting
Local containers	Docker Compose	Local environment with the app and database
Scheduling	In-process timer started from Next.js instrumentation	Scheduled discovery runs
Tests	Node test runner via `tsx --test`	Unit, scheduler, migration, and end-to-end checks

4. System Architecture

Pipeline

```
User preferences (regions, industries, offerings)
        + active offering signals
                  |
                  v
Query generation (query-generation-service.ts)
                  |
                  v
Discovery sources (Google News, PR Newswire, Business Wire)
  - URL normalization, SHA-256 content hashing
  - In-batch and database deduplication
                  |
                  v
AI analyzer (azure-openai-client.ts)
  - Live Azure OpenAI gpt-4o, or local keyword fallback
  - Extracts company, industry, country, signal
  - Classifies problem state and scores the lead
                  |
                  v
Lead service (lead-generation-service.ts)
  - Offering barrier and region barrier
  - Company and offering duplicate check
  - Saves leads to PostgreSQL
                  |
```

```
                  v
search_history statistics, dashboard counts, lead explorer
Digest email (Azure Communication Services) and GET /api/digest
```

How the pieces connect

- **One process does everything.** The scheduler, discovery pipeline, AI calls, and API all run inside the Next.js server. There is no separate worker, queue, or job runner.
- **Startup.** `instrumentation.ts` runs once when the Node runtime starts. It applies database migrations, then starts the scheduler timer.
- **Authentication boundary.** `proxy.ts` wraps all pages and API routes with NextAuth, except sign-in, NextAuth endpoints, and static assets.
- **Layers.** Route handlers authenticate and validate requests, then call services. Services hold the business logic. Repositories in `lib/repositories/` are the only code that writes SQL for each table. Integrations for AI, discovery, and email sit in their own folders.

5. User Roles and Permissions

The application has one role today. Every user who signs in with a Trianz Azure AD account is created with the role `Trianz employee`.

What each user can do

- View and change their own preferences, signals they add, and runs they start.
- View only their own leads, runs, and dashboard counts. Every query is scoped to the signed-in user's ID, and the ID never comes from the request.

What is not in place

- There are no admin, manager, or viewer roles.
- No one can see another user's leads from the portal.
- Signal changes are not restricted to offering owners. Any signed-in user can add or update a signal through `POST /api/signals`, and those changes affect everyone's future searches. This is a known gap; see section 22.

6. Authentication and Authorization

Sign-in

- Sign-in uses NextAuth v4 with the Azure AD provider (Microsoft Entra ID). The configuration is in `lib/auth/auth-options.ts`.
- Sessions use JWT. The database user ID is stored in the token and exposed as `session.user.id`.
- On each sign-in, the user row is created or updated by email, with the role `Trianz employee`.
- A new user gets default preferences: region APAC, industry All industries, offering Concierto Migrate, notification email set to the user's email, schedule 01:30 daily, and scheduled discovery enabled. The 01:30 time is 07:00 IST if the server runs in UTC.

Protecting routes

- `proxy.ts` runs NextAuth's `withAuth` on all paths except `/api/auth/*`, `/auth/signin`, and static assets. Unsigned page requests go to `/auth/signin`.
- Route handlers call `getCurrentWorkspaceUser()` in `lib/workspace/current-user.ts`. It reads the session and loads the user. A request without a session returns `401 UNAUTHORIZED`.

Gaps to know about

- `NEXTAUTH_URL` and `NEXTAUTH_SECRET` are required in production but are not listed in `.env.example`.
- The Azure AD variables are read with non-null assertions, so a missing value fails at request time, not at startup.
- The scheduler and digest routes sit behind the same session check. A service such as Power Automate needs a valid session to call them. The repo has no separate service-to-service token. Confirm how external callers authenticate before relying on them.

7. Core Workflows

Discovery run

1. A run starts from the portal's Discover button, from the scheduler, or from `POST /api/scheduler?force=true`.
2. The run loads the user's preferences and active signals, generates up to 10 queries, fetches articles, deduplicates them, analyzes new articles with AI, saves accepted leads, and records the result in `search_history`.

Reviewing leads

- The dashboard shows lead counts by score, the five most recent leads, and the latest run.
- The lead explorer lists leads with search by company or signal, filters for industry, country, offering, and score, and paging with a limit of 1 to 100.
- Opening a lead shows its full detail and a link to the source article.

Preferences and signals

- Users save regions, industries, offerings, notification email, schedule time, frequency, and day through `POST /api/preferences`.
- Signals are the buying phrases for each offering. They are added with `POST /api/signals`, or through a migration.

Scheduled run

- The scheduler checks every 60 seconds. When a user's time and frequency match, it starts a run for that user. Details are in section 10.

Digest

- After a run that finds new leads, a digest email is sent to the user's notification address.
- `GET /api/digest` returns the same summary as HTML for Power Automate to forward.

8. Pipeline, Step by Step

Query Generation

Query generation is in `lib/services/query-generation-service.ts`. It does not use one template. For each signal, four strategies run in order:

3. `"{signal} {industry} enterprise {geo}"`, for up to two industries and two regions. This is the most specific query.
4. `"{signal} {first industry} enterprise"`.
5. `"{signal} enterprise {geo}"`, using the first region.
6. `"{signal} enterprise"`, a global sweep.

The geo text comes from the region. For example, EMEA maps to `Europe OR Middle East OR UK OR Germany OR UAE`. Duplicate query text is removed and the list is capped at 10 queries per run. If the user has no active signals for their offerings, the discovery service uses a fallback list of queries for each offering.

Discovery and Deduplication

- Each query runs against each source in sequence. A failing query or source is logged and skipped.
- URLs are normalized: the host is lowercased, default ports are removed, tracking parameters (`utm_*`, `fbclid`, `gclid`, `msclkid`, and similar) are removed, remaining parameters are sorted, and the fragment is dropped.
- A content hash is a SHA-256 of the normalized title, the normalized canonical URL, and the first 250 characters of the normalized snippet.
- Deduplication runs in three steps: a duplicate within the same run, a match on the content hash in the database, and a match on the canonical URL in the database.
- New articles are saved to `articles`, linked to the run.
- If nothing new is found, the run re-analyzes up to 40 existing articles against the current preferences, so a preference change still produces results.

AI Analysis and Scoring

- Each candidate article is analyzed one at a time by `azureOpenAiClient.analyzeArticle`.
- With Azure OpenAI configured, the article is sent to the gpt-4o deployment in JSON mode with a temperature of 0.1 and a 25-second timeout. The prompt names the five Concierto offerings, describes the customer pains each one addresses, and asks the model to identify the buyer, the problem state, the offering, the score, and any disqualifying facts.
- Problem states are A (facing the problem), B (actively solving it), and C (already solved). A state C article is redirected to the next-phase offering, its score is reduced by 10, and the cap is Warm.
- Scores are labeled Very Hot (85 to 100), Hot (70 to 84), or Warm (50 to 69).
- Without Azure credentials, a keyword analyzer runs instead. It is for offline development. It cannot reliably separate buyers from vendors, and its country default is weak.
- An article the model rejects, or that errors, produces no lead, and the run continues.

Lead Service, Isolation, and Persistence

`processArticlesIntoLeads` in `lib/services/lead-generation-service.ts` applies these checks to each analyzed article, in order:

7. The article must be a valid opportunity, with `industryValidated` true.
8. The offering must be one of the user's offerings.
9. The country must fall in one of the user's regions. See section 11.
10. The company and offering pair must not already exist for this user, either earlier in the run or among the last 1,000 leads in the database.

Accepted leads are saved to `leads`. Each lead is keyed by user and content hash, so the same article cannot create two leads for one user.

Search History and Dashboard Counts

- Each run writes one `search_history` row. It starts as `running`, then is marked `completed` with the counts of articles found, duplicates removed, and leads generated, or `failed` with the error message.
- Dashboard counts and the run list are read from these rows and from `leads`.

9. Discovery Sources

Source	Adapter file	How it works
Google News	`lib/discovery/sources/google-news.ts`	Queries the Google News RSS search endpoint with a 10-second timeout. Titles in the form "Headline - Publisher" are split to find the publisher. Snippets are cut to 500 characters.
PR Newswire	`lib/discovery/sources/pr-newswire.ts`	RSS adapter for press releases.
Business Wire	`lib/discovery/sources/business-wire.ts`	RSS adapter for press releases.

The RSS parser is a hand-written regular expression parser for `<item>` blocks. Any source that fails returns no articles and the run continues. The source names are part of a fixed type in `lib/discovery/sources/types.ts`. The `RSS_SOURCES` and `GOOGLE_NEWS_LANGUAGE` variables in `.env.example` are not read by the code; the source list and the Google News locale are set in code.

10. Scheduler Internals

How it runs

- The scheduler is `lib/services/scheduler-service.ts`. It is an in-process timer, not a cron job or a separate worker.
- It starts when the app starts. A first check runs 5 seconds later, then one check runs every 60 seconds.
- Both timers are unreferenced, so they do not keep the process alive by themselves.

Each tick

11. Load every preference with scheduled discovery enabled.
12. For each user, compare the saved schedule time with the server's local time, in `HH:mm`. Skip the user if they differ.

13. Build a run key of user ID, date, and time (`userId:YYYY-MM-DD:HH:mm`). If that key is already in the in-memory set, skip the user.
14. Check the frequency rule. Skip the user if it is not due.
15. Add the key to the in-memory set, then start a run for the user.

Frequency rules

Frequency	Rule
Daily	Runs at the scheduled time every day.
Weekly	Runs only on the scheduled weekday (0 is Sunday, 1 is Monday by default), and at least 7 days after the last completed run.
Biweekly	Runs only on the scheduled weekday, and at least 14 days after the last completed run.

If there is no earlier completed run, the first matching day runs.

Baseline reset

The last-run check counts only completed runs that started after the user's preferences were last updated. Saving a new schedule therefore starts the cadence fresh, so the next matching slot fires without waiting out an older interval.

Fail-open

If the database check for the last run fails, the run goes ahead as if it were the first run. This means a short database problem cannot silently stop discovery. The cost is that a weekly user may get an extra run during an outage.

Limitations

- **Timer runs only while the app is running.** If the App Service instance stops, scales to zero, is redeployed, or crashes, scheduled runs do not happen. Missed runs are not replayed.
- **Duplicate lock resets on restart.** The run-key set is kept in memory and is empty after a restart. A restart within the same scheduled minute can start the same run again. The set is also never cleared, so it grows by one key per user per scheduled day.
- **Multiple instances run multiple timers.** The in-memory lock does not coordinate across instances. Keep the App Service plan at one instance unless a shared lock is added.
- **Server time zone.** The schedule compares against the server's local clock. On App Service this is UTC, so 01:30 means 07:00 IST.

External trigger

- `GET /api/scheduler` starts the timer if it is not running and returns status: whether it is running, the last tick time, the last run key, and the active schedules.
- `POST /api/scheduler` runs one evaluation now using the same time and frequency rules.
- `POST /api/scheduler?force=true` runs discovery immediately for every scheduled user. It ignores both the time and the frequency rule, so it should be used only for testing or recovery.
- All three routes require a valid session.

11. Isolation Rules

Isolation is enforced twice. The AI prompt asks the model to respect the user's offerings and regions, and the code checks the result again afterwards. The code check is the one that decides.

Offering Barrier

A lead is saved only when its offering is one of the user's selected offerings. The five offerings are:

- **Concierto Migrate:** moving on-premises or legacy infrastructure to the cloud.
- **Concierto Modernize:** re-architecting applications for cloud-native operation.
- **Concierto Manage:** managed cloud operations, FinOps, governance, and reliability.
- **Concierto Maximize:** getting value and return from cloud investment.
- **Concierto Intelligence:** AI, data, and analytics transformation.

Offering names are a fixed set in the code. A signal tied to any other name can never produce a lead. For a user with an empty offerings list, the check allows all offerings, but the default preferences always set at least one.

Region Barrier

The region check is in `lib/domain/regions.ts`.

16. If the user's regions are empty or include Global, every country passes.
17. Otherwise the country is mapped to a region. The code first looks for an exact country name match, then for keywords such as city and variant spellings. A country that matches nothing is mapped to Global.
18. The country passes if its region is one of the user's regions.

Regions and their countries:

Region	Countries
North America	United States, Canada, Mexico
APAC	Singapore, Japan, Australia, India, China, Hong Kong, New Zealand, South Korea, Indonesia, Malaysia, Philippines, Thailand, Taiwan, Vietnam
EMEA	United Kingdom, Germany, France, Netherlands, Switzerland, Spain, Italy, Sweden, Ireland, UAE, Saudi Arabia, Poland, Belgium, Norway, Denmark, Finland, Austria, South Africa
Global	Matches every region

Example: a user who monitors APAC and North America gets an article about a UK bank. The country maps to EMEA, which is not one of the user's regions, so the lead is dropped.

Rejection Behavior

- Rejection is silent. No row is written to `leads`.
- The article row remains in `articles`, so it is not treated as new on the next run, and it still counts toward articles found.
- The reason for rejection is not stored. The model's reason is saved only for accepted leads.
- Duplicate company and offering skips are written to the console log. Other rejections are not logged.
- To investigate a missing lead, check the article row and rerun the analysis for it.

12. Data Model

Tables

Table	Purpose
users	One row per signed-in person: email, name, role, timestamps.
preferences	One row per user: regions, industries, offerings, notification email, schedule time, frequency, day, enabled flag, workspace name.
offering_signals	Buying signals for each offering, with a priority from 1 to 1000. Drives query generation.
search_history	One row per discovery run, with status, timing, counts, sources, and queries.
articles	Each unique article collected, with its canonical URL and content hash.
leads	Qualified opportunities, one per article per user.
schema_migrations	Applied migration names and checksums.

Migrations

Migrations are SQL files in db/migrations/, applied in filename order.

File	Change
001_initial_schema.sql	Creates the six core tables, indexes, and the updated_at triggers. Preferences start with singular region, industry, and offering columns.
002_reference_offering_signals.sql	Seeds an early signal set that includes a Concierto Cloud offering.
003_multi_value_preferences.sql	Renames the preference columns to regions, industries, and offerings, and converts them to text arrays with non-empty checks.
003_update_offering_signals.sql	Removes the 002 signals and seeds signals for the five Concierto offerings.
004_fix_offering_signals.sql	Reinserts the five-offering signals.
004_preference_notification_email.sql	Adds the notification email to preferences, filled from the user's email.
005_force_upsert_offering_signals.sql	Reinserts the five-offering signals again. Safe to rerun.
006_schedule_frequency.sql	Adds the schedule frequency, default daily.
007_schedule_day.sql	Adds the schedule day, default Monday (1).

Two files share the 003 prefix and two share 004. They run in alphabetical order, which is safe because each is idempotent or independent. Do not add a new file with a duplicate prefix.

Migration Runner

The runner is lib/db/migrate.ts.

19. It creates schema_migrations if needed.
20. It takes a PostgreSQL advisory lock, so two instances cannot migrate at the same time.
21. For each file, it computes a SHA-256 checksum. A file already applied with the same checksum is skipped.
22. If an applied file has a different checksum, the runner stops with an error. Applied migrations must never be edited.

23. A new file is run inside a transaction and its checksum is recorded.

The runner starts from `instrumentation.ts`. If it fails, the error is logged and the app still starts. Check the startup log after every deploy.

13. Database Schema Reference

users

Column	Notes
`id`	UUID primary key
`email`	Unique
`name`	Display name
`role`	Default `Trianz employee`
`created_at`, `updated_at`	Timestamps

preferences

Column	Type	Notes
`user_id`	UUID, unique, foreign key to users	One row per user
`regions`	Text array, not empty	Regions to monitor
`industries`	Text array, not empty	Industries to monitor
`offerings`	Text array, not empty	Concierto offerings to monitor
`notification_email`	Text, not null	Digest recipient
`schedule_time`	Time	Server local time, `HH:mm`
`schedule_frequency`	Text, default `daily`	`daily`, `weekly`, or `biweekly`
`schedule_day`	Integer, default 1	0 (Sunday) to 6 (Saturday)
`scheduled_discovery_enabled`	Boolean, default true	Turns scheduling on or off
`workspace_name`	Text, default `Trianz GTM`	Used in the digest
`updated_at`	Timestamp	Maintained by trigger; the scheduler's baseline

offering_signals

Column	Notes
`id`	Primary key
`offering`	One of the Concierto offerings
`signal`	The buying phrase
`priority`	1 to 1000, default 100; lower runs first
`is_active`	Only active signals are used
Timestamps	Created and updated

Unique on the offering and signal, and unique on the offering and the lowercased signal.

search_history

Column	Notes
`id`, `user_id`	Primary key and owner
`started_at`, `completed_at`	Run timing
`articles_found`, `duplicates_removed`, `leads_generated`	Run counts

status	running, completed, or failed
error_message	Set when the run fails
sources, search_queries	Text arrays of what the run used

articles

Column	Notes
id, search_history_id	The run that found it; cleared if the run is deleted
source, title, url, canonical_url, snippet	Article details
published_at, collected_at	Dates
content_hash	Unique

leads

Column	Notes
id, user_id, article_id, search_history_id	Keys; article and run links are cleared on delete
company, initials, industry, country, signal, offering	From the analyzer
source, source_url	Publisher and article link
score_label	Very Hot, Hot, or Warm
score_value	0 to 100
summary, reason	Analyzer output
project_stage, business_impact, procurement_activity	Sales context
industry_validated	True for saved leads
content_hash	Unique together with the user ID
published_at, detected_at	Dates

Indexes cover the user with score, the user with detected date, the user with industry, and the foreign keys.

14. API Reference

Routes

All routes require a NextAuth session, except `/api/auth/*`.

Method	Path	Purpose
GET	/api/dashboard	Lead counts, recent leads, latest run, scheduling status
GET	/api/leads	Filtered, paged lead list
GET	/api/leads/:id	One lead
GET	/api/preferences	The user's preferences
POST	/api/preferences	Create or update preferences
POST	/api/discover-leads	Start a discovery run now
GET	/api/search-history	Recent runs
GET	/api/signals	Signals, optionally filtered by offering
POST	/api/signals	Add or update a signal
GET	/api/scheduler	Scheduler status
POST	/api/scheduler	Run one scheduler evaluation; ?force=true runs every scheduled user
GET	/api/digest	HTML digest of recent leads

GET	/api/workspace	The signed-in user's profile
PATCH	/api/workspace	Update the signed-in user's name and role
GET	/api/healthz	Health check

Request and Response Shapes

Successful responses have the form `{ "data": ... }`. Errors have the form `{ "error": { "code": "...", "message": "..." } }`.

GET /api/leads takes these query parameters: `q` (company or signal text), `industry`, `country`, `offering`, `score` (repeatable: Very Hot, Hot, Warm), `limit` (1 to 100), and `offset` (0 to 10000). It returns items, total, limit, and offset.

GET /api/leads/:id returns one lead.

POST /api/preferences accepts `regions`, `industries`, `offerings`, `notificationEmail`, `scheduleTime`, `scheduledDiscoveryEnabled`, `workspaceName`, and also `scheduleFrequency` and `scheduleDay`. The API contract document does not list the last two yet.

POST /api/discover-leads starts a run for the current user and returns its result. A run can take several minutes because articles are analyzed one at a time.

GET /api/search-history takes `limit` (1 to 100).

GET /api/signals returns all signals. With `offering` values, it returns active signals for those offerings sorted by priority.

POST /api/signals accepts `offering`, `signal`, and `priority`, and updates an existing signal with the same offering and text.

GET /api/scheduler returns `isRunning`, `lastTickAt`, `lastRunKey`, `activeSchedulesCount`, and the schedules.

POST /api/scheduler returns a status for each user: `triggered`, `skipped`, or `already_ran`. With `force=true` it returns `forced: true` and the results.

GET /api/digest returns HTML, not JSON. The portal link in it defaults to `http://localhost:3000` unless configured.

GET /api/workspace returns the signed-in user. **PATCH /api/workspace** accepts `name` and `role`.

Error Codes

Code	HTTP status	Meaning
UNAUTHORIZED	401	No valid session
USER_NOT_FOUND	404	The session's user no longer exists
PREFERENCES_NOT_FOUND	404	The user has no saved preferences
LEAD_NOT_FOUND	404	No lead with this ID for this user
VALIDATION_ERROR	400	The request failed validation

Other codes are defined in `lib/http/response.ts`.

15. Email and Digest

- `lib/services/email-service.ts` sends the digest through Azure Communication Services using the connection string in the environment.
- A digest is sent after a run that produces leads. It goes to the user's notification email.
- If the connection string is missing, the recipient is invalid, or there are no leads, the send is skipped and a status line is logged.
- The send is not awaited. If it fails, the error is logged to the console only. The run still reports success, and no delivery record is kept.
- The sender address is set in the code, not in the environment. Changing it needs a code change.
- `GET /api/digest` produces the same content as HTML for Power Automate to forward on a schedule. Its portal link defaults to localhost, so set the correct URL before sharing it.

16. Setup and Local Run

Prerequisites

- Node.js 20 or 22 LTS
- PostgreSQL 16 or later, local or in Azure
- Access to a Trianz Azure AD app registration, for sign-in
- Optional: Azure OpenAI and Azure Communication Services, for live analysis and email

Steps

24. Install dependencies with `npm install`.
25. Copy `.env.example` to `.env.local` and fill in the values. See section 17 for the variable names.
26. Apply migrations with `npm run db:migrate`. Migrations also run when the app starts.
27. Start the app with `npm run dev`, then open `http://localhost:3000`.
28. Sign in, save preferences, and start a manual discovery run.

Without the Azure AD values, sign-in cannot complete. Without the Azure OpenAI values, the keyword fallback analyzer runs and results are not representative.

Scripts

Script	Purpose
`npm run dev`	Development server with Turbopack
`npm run build`	Production build
`npm start`	Production server
`npm run typecheck`	Type check without output
`npm test`	Unit tests
`npm run test:db`	Migration tests; needs a test database
`npm run test:discovery`	Discovery tests
`npm run test:ai`	AI analyzer tests
`npm run test:e2e`	End-to-end script
`npm run test:azure`	Live Azure OpenAI connection check
`npm run db:migrate`	Apply migrations

17. Environment Variables Reference

Only names are listed here. Values belong in `.env.local` locally and in App Service configuration in Azure. Never commit values.

Database

Variable	Required	Notes
DATABASE_URL	Yes	PostgreSQL connection URL
DATABASE_SSL	Yes in Azure	Turn on SSL; on by default unless the host is local
DATABASE_SSL_REJECT_UNAUTHORIZED	Yes in Azure	Keep true in production
DATABASE_POOL_MAX	No	Connection pool size, default 10
DATABASE_IDLE_TIMEOUT_MS	No	Default 30000
DATABASE_CONNECTION_TIMEOUT_MS	No	Default 10000
DATABASE_TEST_URL	Tests only	Must never point at production

Authentication

Variable	Required	Notes
AZURE_AD_CLIENT_ID	Yes	App registration client ID
AZURE_AD_CLIENT_SECRET	Yes	Client secret; store in App Service settings or Key Vault
AZURE_AD_TENANT_ID	Yes	Tenant ID
NEXTAUTH_URL	Yes in production	Public URL of the app; not in `.env.example`
NEXTAUTH_SECRET	Yes in production	Session signing secret; not in `.env.example`

Azure OpenAI

Variable	Required	Notes
AZURE_OPENAI_ENDPOINT	For live analysis	Resource endpoint
AZURE_OPENAI_API_KEY	For live analysis	Without it, the keyword fallback runs
AZURE_OPENAI_DEPLOYMENT	No	Default `gpt-4o`
AZURE_OPENAI_API_VERSION	No	Default `2024-10-21`

Discovery

Variable	Required	Notes
RSS_SOURCES	Not used	In `.env.example`, but the code does not read it
GOOGLE_NEWS_LANGUAGE	Not used	In `.env.example`, but the code does not read it

Email

Variable	Required	Notes
AZURE_COMMUNICATION_CONNECTION_STRING	For digest email	Without it, the digest is skipped
AZURE_COMMUNICATION_SENDER_EMAIL	Not used	In `.env.example`, but the sender is set in code

Obsolete

Variable	Notes
`WORKSPACE_USER_EMAIL`, `WORKSPACE_USER_NAME`, `WORKSPACE_USER_ROLE`	Replaced by the signed-in session user. Remove them.

18. Deployment and Azure Infrastructure

Azure Setup

Production runs on:

- **Azure App Service** on Linux with the Node runtime. Run one instance (see section 10).
- **Azure Database for PostgreSQL Flexible Server** for the application database.
- **Azure OpenAI** with a gpt-4o deployment.
- **Azure Communication Services** for email.
- **Microsoft Entra ID** (Azure AD) app registration for sign-in.

Resource names, hostnames, and subscription details are not in this document. Ask the platform owner for them.

Deployment steps:

29. Create the App Service and set the variables from section 17 in its configuration.
30. Set the App Service plan to one instance.
31. In the Azure AD app registration, add the redirect URI `<app-url>/api/auth/callback/azure-ad`.
32. Deploy the code. Migrations run at startup.
33. Check the startup log for the migrations-complete message and the scheduler-started message.
34. Sign in, save preferences, and run discovery once manually.

Company Laptop Setup

- Copy the repository without `node_modules`, `.next`, and `.env.local`. Create a new `.env.local` with the company values.
- Outbound HTTPS must reach Azure OpenAI, Azure Communication Services, Google News, PR Newswire, and Business Wire. A proxy that blocks the news sites produces zero articles without an error message.
- If the proxy blocks Azure OpenAI, live analysis fails for each article. The fallback analyzer is not used automatically in that case.
- The full checklist is in `docs/COMPANY_LAPTOP_DEPLOYMENT.md`.

19. Developer Guide

Add a Discovery Source

35. Create a source class in `lib/discovery/sources/` that returns collected articles for a query.
36. Add its name to the source type in `lib/discovery/sources/types.ts`.
37. Export it from `lib/discovery/sources/index.ts`.
38. Add it to the default source list in `discovery-service.ts`.

39. Normalize every article URL with the same normalization used by deduplication.
40. Add a parser test under `tests/discovery/`.

Add a Buying Signal

Signals are stored in the database, so no deploy is needed.

- Use the API: `POST /api/signals` with offering, signal text, and priority.
- Or add an insert in a new migration file. Use `ON CONFLICT` so it can be rerun.

Priority ranges from 1 to 1000, and lower numbers run first.

Add a Migration

41. Create `db/migrations/NNN_description.sql` with the next number. Use lowercase letters, digits, and underscores.
42. Write plain SQL, and make it safe to rerun where practical.
43. Never edit a migration that has already been applied. The checksum check will block startup.
44. Run `npm run db:migrate` locally, then `npm run test:db` against a test database.
45. Update the schema section of this document.

Add an API Route

46. Create `app/api/<name>/route.ts`, with the Node runtime and dynamic rendering set.
47. Resolve the user with `getCurrentWorkspaceUser()`. Never take a user ID from the request.
48. Validate input with the helpers in `lib/http/validation.ts`.
49. Return the success and error helpers from `lib/http/response.ts`.
50. Add the route to section 14 and to `docs/api-contract.md`.

Adding an Offering

An offering affects more than the database. It touches the offering type and the system prompt in the AI client, the fallback analyzer, the fallback query list in discovery, the signals in a migration, and the offering choices in the portal UI.

20. Codebase Structure

Folder or file	Contents
`app/`	Pages and API route handlers (`app/api/**/route.ts`)
`components/`	Portal UI components
`lib/auth/`	NextAuth configuration
`lib/ai/`	Azure OpenAI client and keyword fallback analyzer
`lib/config/`	Environment reading and validation
`lib/db/`	Connection pool and migration runner
`lib/discovery/`	News sources, RSS parsing, and source types
`lib/domain/`	Types, mappers, and region data
`lib/http/`	Response and validation helpers
`lib/repositories/`	SQL for each table
`lib/services/`	Discovery, query generation, deduplication, lead generation, scheduler, email
`lib/workspace/`	Resolving the current user from the session
`db/migrations/`	SQL migrations, applied in order
`tests/`	Unit, scheduler, migration, discovery, digest, and end-to-

	end tests
`docs/`	API contract, technical reference, deployment notes, handoff notes
`proxy.ts`	Authentication middleware
`instrumentation.ts`	Startup hook: migrations, then scheduler
`docker-compose.yml`	Local containers
`.env.example`	Variable names with placeholder values

Layering rule: route handlers call services, services call repositories, and only repositories run SQL.

21. Testing

Unit Tests

Run with `npm test`. The unit tests cover the lead generation filters, deduplication and URL normalization, query generation, the discovery flow, signal mapping, and the digest HTML. The README says 23 unit tests. Run `npm test` for the actual count, since it changes as tests are added.

Scheduler Tests

`tests/scheduler/scheduler.test.ts` covers time matching, the daily, weekly, and biweekly rules, and the duplicate-run lock.

Migration Tests

`tests/db/migrations.test.ts` runs the migration runner against a test database. Run it with `npm run test:db`. It needs `DATABASE_TEST_URL`, must never point at production, and is skipped when the variable is missing.

End-to-End Tests

`tests/phase5-e2e.ts` runs a script-level flow through the system. Run it with `npm run test:e2e`. It is not part of `npm test`.

Not Covered

- The live Azure OpenAI path. Use `npm run test:azure` manually.
- The real news feeds, which change over time.
- Email delivery.
- The dashboard UI.

22. Known Issues and Gaps

Data and correctness

51. **Seed data may contain an old offering name.** Migration 002 seeds a Concierto Cloud signal, and migration 003 removes it. Check that no active signal uses a name outside the five offerings.
52. **Rejected leads leave no record.** Offering and region rejections are silent, which makes missing leads hard to explain.

53. **Unmapped countries fall to Global.** A new country spelling is rejected for every user who did not choose Global.
54. **Duplicate check covers only 1,000 leads.** Older duplicates are not caught.
55. **The fallback analyzer is not an AI model.** Do not judge lead quality from runs without Azure credentials.

Scheduler and operations

56. **Scheduled runs stop when the app stops.** Missed runs are not replayed.
57. **The duplicate lock resets on restart.** A restart in the same minute can start a run twice.
58. **Multiple instances run multiple timers.** Keep one instance or add a shared lock.
59. **Digest sending is not tracked.** A failed send is only logged.
60. **Forced scheduler runs ignore the schedule.** `POST /api/scheduler?force=true` starts discovery for every scheduled user at once.
61. **Migration failure does not stop startup.** The app can start with an incomplete schema.
62. **AI analysis runs one article at a time.** Large runs are slow, and the request that started them may time out.

Security and configuration

63. **No role-based access.** Every user has the same role. Any signed-in user can change signals, which affect everyone.
64. **Auth variables are missing from `.env.example`.** `NEXTAUTH_URL` and `NEXTAUTH_SECRET` are not listed.
65. **Unused variables in `.env.example`.** `RSS_SOURCES`, `GOOGLE_NEWS_LANGUAGE`, `AZURE_COMMUNICATION_SENDER_EMAIL`, and the `WORKSPACE_USER_*` variables are not read by the code.
66. **Sender address is hard-coded.** The email sender is set in code.
67. **Digest link defaults to localhost.** Set the portal URL before sharing digests.
68. **External callers need a session.** It is not clear how Power Automate should authenticate to the scheduler and digest routes.

Documentation drift

69. **Migration count.** The README says to apply 001 to 004. The repo has 001 to 007.
70. **Offering count.** The README and handoff describe three offerings. The code has five.
71. **Authentication.** Older docs describe the placeholder workspace user. The app uses Azure AD through NextAuth.
72. **News sources.** Older docs list Google News only. The code also uses PR Newswire and Business Wire.
73. **Test count.** The README says 23 unit tests. Verify with `npm test`.
74. **Query format.** Older docs show a simple signal, industry, and region query. The real queries use the four strategies in section 8.

23. What's Left to Do

- Store scheduler run records and use a shared lock, so schedules survive restarts and scale-out.
- Record a reason for every rejected article.
- Await the digest send and record its delivery status, with retry.
- Analyze articles in parallel with a concurrency limit.
- Add roles, and restrict signal changes and scheduler triggers to an admin role.
- Read the sender address and sources from the environment, and remove unused variables.
- Show unknown countries to the user instead of mapping them to Global.

- Document the service authentication path for external triggers.
- Update the README and handoff to match the current migrations, offerings, sources, and authentication.

24. Glossary

Term	Meaning
Lead	A qualified enterprise opportunity for one user, from one article.
Signal	A buying-intent phrase tied to an offering, such as "data center exit".
Offering	One of the five Concierto products the user monitors.
Region	APAC, EMEA, North America, or Global. Each has a fixed country list.
Score	Very Hot (85 to 100), Hot (70 to 84), or Warm (50 to 69).
Problem state	A: facing the problem. B: actively solving it. C: already solved, so redirected to the next offering.
Discovery run	One pass of query, fetch, deduplicate, analyze, and save, recorded in search history.
Content hash	SHA-256 of the normalized title, canonical URL, and snippet. Detects duplicate articles and leads.
Canonical URL	The article URL with tracking parameters removed and parameters sorted.
Fallback analyzer	The keyword-based analyzer used without Azure OpenAI.
Digest	The summary email, and the matching HTML feed, of a run's new leads.
Fail-open	When the scheduler's last-run check fails, the run goes ahead.

25. Summary

The GTM Lead Intelligence Agent turns public news into scored, region-filtered sales leads for Trianz's five Concierto offerings. It runs as one Next.js application on Azure App Service, with PostgreSQL for storage, Azure OpenAI for analysis, Azure AD sign-in, and Azure Communication Services for digest email.

The system works end to end: signals become queries, articles are collected and deduplicated, the AI analyzer scores each opportunity, and the lead service enforces offering and region rules before saving. The main risks are operational. Scheduled runs depend on one live process, the duplicate lock does not survive restarts, digest delivery is not tracked, and the documentation has drifted from the code. Section 22 lists the gaps, and section 23 lists the work to close them.